

1. Backend and Frontend

For this assignment, I used Flask for the backend and JavaScript for the frontend. Flask reads the CSV file, converts the data into usable values, calculates acceleration magnitude, and sends the data through the `/api/telemetry` endpoint. JavaScript gets this data and uses Plotly to display the graphs.

I separated it this way so that Python handles the telemetry data while JavaScript handles how it is displayed. With a real CanSat, I could replace the CSV input with data coming from a radio or serial connection while keeping most of the frontend the same.

2. Measurements at Different Rates

Each telemetry sample has a timestamp, which I use as the x-axis of the graphs. In the provided CSV, the measurements are already grouped into rows, so I do not have to deal with sensors sending data at different times.

With a real CanSat, sensors could update at different rates. I would timestamp incoming measurements and update each plot separately when new data for that sensor arrives. This would let a faster sensor update without having to wait for the other sensors.

3. Acceleration Magnitude

I calculate acceleration magnitude in the Flask backend using:

$$a = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

I did this in the backend because Python is already reading and processing the accelerometer values from the CSV. This keeps the calculations in one place and leaves JavaScript mainly responsible for displaying the data.

The advantage is that the frontend is simpler and every client receives the same calculated value. The disadvantage is that the backend has to do slightly more processing. Doing the calculation in JavaScript would reduce the backend work, but it would move more data processing into the frontend.

4. Offline and Field Failures

One problem with my current application is that Plotly is loaded from the internet. If the Ground Station is offline, Plotly may not load and the graphs would not work. I could fix this by storing Plotly locally with the project.

The application could also have problems if the CSV is missing or contains invalid data. I could add error handling so that one bad value does not crash the application.

With a real CanSat, the radio could disconnect, packets could be lost, or a sensor could stop sending data. I could use timestamps and timeouts to detect when data has stopped updating and show a warning to the user instead of continuing to display old data as if it were current.

